DATABRICKS MCQ MASTER PREPARATION GUIDE

Syllabus-mapped practice bank • Easy, Medium & Difficult • Conceptual, code-reading and scenario-based questions

How to use this guide

Attempt every question before reading the answer. Each question has four options, the correct answer, and a short explanation. Scenario questions emphasize safe production behavior, idempotency, governance, troubleshooting, and validation.

Syllabus coverage from your image

MCQ difficulty	Topics covered
Easy	Lakeflow; Delta Lake; CDC; SDP/MLflow; Unity Catalog; AI/ML/LLM; data masking; compliance; SQL
Medium	DLT/SDP; CDC; Delta Lake; joins, data quality, permissions and operational scenarios
Difficult	Architecture/cluster scenarios; Delta Lake; CDC; governance, security, production and performance cases

Total practice questions: 89 — This is a broad preparation bank based on the topics shown, not a guarantee of the exact exam questions. Platform features and names can evolve; verify any version-specific behavior in your workspace documentation.

Quick coverage checklist

- ■ Lakeflow (Jobs, dependencies, monitoring, retries)
- ■ Delta Lake (ACID, MERGE, history/time travel, schema, OPTIMIZE/VACUUM)
- ■ CDC (watermarks, ordering, duplicates, deletes, late events)
- ■ SDP / declarative pipelines (expectations, dependencies, incremental processing)
- ■ Unity Catalog (catalog.schema.table, grants, external locations, secrets)
- ■ AI/ML/LLM (MLflow, overfitting, RAG, evaluation, privacy)
- ■ Data masking (policy enforcement, least privilege, re-identification)
- ■ Compliance (audit, retention, secure access, deletion, residency)
- ■ SQL (WHERE/HAVING, duplicates, window functions, joins, NULL, UNION)
- ■ Architecture/cluster (driver/executor, broadcast, shuffle, skew, transformations/actions)

Lakeflow

Q1. [Easy] In Databricks, Lakeflow is best understood as:

A. A single SQL function
B. A data engineering solution family for building and operating pipelines and workflows
C. A file compression algorithm
D. A replacement for Unity Catalog

Answer: B — Lakeflow brings together capabilities for data ingestion, pipeline development, and orchestration; it is not one SQL function.

Q2. [Easy] A team wants to schedule a notebook and coordinate dependent tasks. Which capability is most relevant?

A. Lakeflow Jobs
B. Delta time travel
C. Column masking
D. MLflow model signature

Answer: A — Lakeflow Jobs orchestrates tasks, dependencies, schedules, retries, and monitoring.

Q3. [Easy] Which is the strongest reason to define task dependencies in a workflow?

A. It makes every task run simultaneously
B. It guarantees zero failures
C. It ensures downstream tasks wait for required upstream tasks
D. It removes the need for data validation

Answer: C — Dependencies encode execution order so downstream tasks do not run before prerequisites succeed.

Q4. [Medium] A daily workflow sometimes runs the Gold aggregation before the Silver load finishes. What is the best fix?

A. Increase the notebook font size
B. Add an explicit task dependency from Silver to Gold
C. Disable logging
D. Convert all tables to CSV

Answer: B — The workflow should express the true dependency rather than relying on timing or task order assumptions.

Q5. [Medium] A job failed after partially writing output. Before retrying, what should the engineer verify first?

A. Whether the pipeline is idempotent and what was already committed
B. Whether the notebook has enough comments
C. Whether the table name is short
D. Whether the driver has a browser

Answer: A — Safe retries require understanding partial writes, batch identity, checkpoints, and duplicate risk.

Q6. [Difficult] An ingestion task succeeds but a downstream transformation fails. The source batch must not be loaded twice. Which design is strongest?

A. Always append the entire source again
B. Use a batch identifier and idempotent write/MERGE logic, then retry the failed task safely
C. Delete the whole target table
D. Turn off task dependencies

Answer: B — Batch tracking and idempotent writes make retries safe and help avoid duplicate ingestion.

Q7. [Medium] Which monitoring information is most useful when diagnosing a failed Lakeflow job?

A. Run ID, failed task, error logs, timestamps, and upstream/downstream status
B. Notebook color theme only
C. Number of browser tabs
D. User profile picture

Answer: A — Run-level evidence narrows the failure and reveals impact on dependent tasks.

Q8. [Difficult] A job is scheduled every hour, but one run may last longer than an hour. What should be assessed?

A. Whether overlapping runs can cause contention or duplicate writes
B. Whether SQL keywords are uppercase
C. Whether the catalog has a long name
D. Whether comments use emojis

Answer: A — Concurrency, job settings, idempotency, and target-table write conflicts should be considered.

Q9. [Difficult] A job succeeds technically, but the Gold table is stale because the upstream ingestion task processed no new files. What control helps catch this?

A. Data freshness/volume checks and alerts
B. Change the job name
C. Remove all dependencies
D. Disable run history

Answer: A — Operational success does not guarantee business correctness; freshness and volume checks detect silent failures.

Delta Lake

Q10. [Easy] What does Delta Lake add on top of Parquet files?

A. Only a graphical user interface
B. A transaction log and table-management features such as ACID transactions and time travel
C. Automatic business-key uniqueness
D. A requirement to use only Python

Answer: B — Delta uses Parquet data files plus a transaction log to support reliable table operations.

Q11. [Easy] Which statement about a Delta table is correct?

A. It cannot be queried with SQL
B. It stores data only in row-oriented CSV
C. It can support INSERT, UPDATE, DELETE, and MERGE operations
D. It always enforces primary-key uniqueness

Answer: C — Delta supports transactional DML; it does not automatically enforce uniqueness of arbitrary business keys.

Q12. [Easy] What is Delta time travel used for?

A. Querying an earlier table version
B. Scheduling a cluster shutdown
C. Masking every column
D. Creating a new workspace

Answer: A — Time travel can query retained historical table versions using version or timestamp options.

Q13. [Medium] A MERGE source contains two rows for the same target key, and both match one target row for update. What is the safest response?

A. Run MERGE repeatedly
B. Deduplicate or otherwise reduce the source to one deterministic row per key before MERGE
C. Remove the ON condition
D. Convert the target to JSON

Answer: B — Multiple source matches can make a MERGE ambiguous or fail; define a deterministic source rule.

Q14. [Medium] A table receives new columns from an upstream source unexpectedly. Which approach is safest?

A. Enable every schema evolution option globally without review
B. Validate the schema change and use an intentional schema evolution policy
C. Drop all existing columns
D. Ignore data types

Answer: B — Schema enforcement protects table quality; evolution should be explicit and reviewed.

Q15. [Easy] Which operation is commonly used on Databricks to compact many small Delta files?

A. OPTIMIZE
B. VACUUM only
C. DESCRIBE HISTORY only
D. GRANT

Answer: A — OPTIMIZE can compact small files and improve query performance where appropriate.

Q16. [Medium] What is a key risk of running VACUUM with an overly aggressive retention period?

A. It may remove files needed for older versions/time travel
B. It automatically creates duplicate keys
C. It converts Delta into a SQL warehouse
D. It adds masking policies

Answer: A — VACUUM removes unreferenced files; retention choices affect historical recovery and concurrent readers.

Q17. [Easy] Which command can show a Delta table's change/version history?

A. DESCRIBE HISTORY table_name
B. SHOW USERS ONLY
C. MASK TABLE
D. START CLUSTER

Answer: A — DESCRIBE HISTORY reports operations and table versions.

Q18. [Medium] A pipeline appends the same batch again after a retry. What is the best long-term prevention?

A. Rely on file names only
B. Design idempotent ingestion using stable keys/batch IDs and MERGE or deduplication
C. Disable transaction logging
D. Increase output partitions indefinitely

Answer: B — Delta transactions do not by themselves prevent duplicate business records.

Q19. [Difficult] Queries scan a large Delta table and many files contain irrelevant date ranges. What should be evaluated first?

A. Data layout, partition/clustering strategy, predicate filters, and query plans
B. Rename every column
C. Disable statistics
D. Replace all data with one CSV

Answer: A — Data skipping and effective layout can reduce scanned data; choose based on actual query patterns.

Q20. [Difficult] A production table needs an update based on a reference dataset. Which sequence is safest?

A. Run MERGE immediately with unvalidated duplicate source keys
B. Validate permissions and source keys, test the MERGE, assess impact, obtain approval, then reconcile results
C. Delete the target table first
D. Turn off audit history

Answer: B — Controlled changes require source validation, test evidence, authorization, and post-write checks.

Q21. [Medium] Which statement correctly compares Parquet and Delta Lake?

A. Parquet is a columnar file format; Delta adds a transaction log and table semantics
B. Delta is always a row-based file format
C. Parquet supports all Delta transaction features by itself
D. They are identical in every respect

Answer: A — Delta commonly stores Parquet files and adds transactional metadata and operations.

Q22. [Medium] Which Delta feature supports reading data as of a retained earlier commit?

A. Time travel
B. Cluster policy
C. Task dependency
D. SQL warehouse sizing

Answer: A — Time travel can query historical versions while required data files remain available.

CDC

Q23. [Easy] What does Change Data Capture (CDC) primarily identify?

A. Changes such as inserts, updates, and deletes in source data
B. Notebook cell colors
C. Only full-table exports
D. Cluster node count

Answer: A — CDC captures changes so downstream systems can process incremental updates.

Q24. [Easy] Why can CDC be more efficient than repeatedly reloading a whole large table?

A. It processes only detected changes when correctly configured
B. It removes the need for data quality checks
C. It guarantees all records are valid
D. It disables transactions

Answer: A — Incremental processing can reduce data movement and compute, while still needing correctness controls.

Q25. [Medium] A CDC stream contains multiple updates for one customer. The target should reflect the latest state. What should you do?

A. Randomly select one event
B. Order by a reliable change sequence/timestamp and apply a deterministic latest-record rule
C. Drop every update
D. Append all events to a current-state table without logic

Answer: B — Ordering and deterministic conflict resolution are necessary when several changes affect one key.

Q26. [Medium] Which source field is especially useful for tracking incremental changes when available?

A. A reliable change version, sequence number, or update timestamp
B. Font size
C. Notebook owner display name
D. Random comment text

Answer: A — A trustworthy watermark or source sequence helps identify new/changed records.

Q27. [Difficult] A CDC feed can deliver events late or out of order. Which design best protects the current-state table?

A. Apply every arriving event regardless of sequence
B. Use source sequence/version checks, deduplication, and explicit late-event handling
C. Drop all events after midnight
D. Run a full reload for each event

Answer: B — Version-aware merge logic prevents older events from overwriting newer state.

Q28. [Medium] How should deletes from a source be represented downstream?

A. Ignore them in every case
B. Apply a defined delete/tombstone policy so the target reflects source semantics
C. Convert them to inserts
D. Rename the deleted row

Answer: B — Delete handling must be part of the CDC contract, often through delete events or tombstones.

Q29. [Difficult] A CDC pipeline's watermark advances before the target write commits. A failure may skip changes. What is the better design principle?

A. Advance state only in a coordinated, recoverable way after successful processing
B. Always advance the watermark first
C. Remove checkpoints
D. Ignore the failed batch

Answer: A — Checkpoint/watermark management must prevent skipped data and support safe recovery.

Q30. [Easy] What is a common CDC use case?

A. Keeping a warehouse current with source-system changes
B. Designing notebook backgrounds
C. Setting SQL capitalization
D. Compressing images

Answer: A — CDC is commonly used for near-real-time or incremental synchronization.

Q31. [Difficult] A source emits the same CDC event twice with the same event ID. What should downstream processing ideally do?

A. Deduplicate using a stable event identifier or idempotent key
B. Count it as two distinct business updates always
C. Delete the whole table
D. Ignore source event IDs

Answer: A — Stable event IDs help make CDC ingestion idempotent.

SDP / Spark Declarative Pipelines

Q32. [Easy] What is the central idea of Spark Declarative Pipelines (SDP)?

A. Define datasets and transformations declaratively while the platform manages pipeline execution
B. Manually schedule every Spark task with shell commands
C. Replace SQL with spreadsheets
D. Disable data quality

Answer: A — Declarative pipelines let developers specify datasets and logic while the pipeline engine manages execution.

Q33. [Easy] In a declarative pipeline, what is the purpose of a data-quality expectation?

A. Define a rule that records must satisfy, with configured handling for violations
B. Increase cluster color contrast
C. Grant all users admin access
D. Force all values to strings

Answer: A — Expectations express data-quality constraints; behavior may warn, drop, or fail depending on configuration.

Q34. [Medium] A pipeline must retain invalid records for investigation instead of silently discarding them. What is a good design?

A. Capture/quarantine invalid rows and record the reason
B. Delete the source files immediately
C. Disable expectations
D. Convert every invalid value to 0 without logging

Answer: A — Quarantine and reject tracking preserve evidence and make data quality observable.

Q35. [Medium] Which statement best describes declarative pipeline dependencies?

A. The engine can derive execution order from dataset dependencies
B. Dataset dependencies are irrelevant
C. Every table must run manually in a fixed time
D. Dependencies disable lineage

Answer: A — The pipeline graph expresses relationships between datasets and helps manage execution order.

Q36. [Medium] A source table is updated continuously and the downstream table must be maintained incrementally. Which feature is most relevant?

A. Streaming/incremental table processing where supported
B. Manual CSV downloads after each row
C. Disabling checkpoints
D. Running collect() on all records

Answer: A — Incremental or streaming processing can update outputs as new data arrives, subject to pipeline design.

Q37. [Difficult] A pipeline passes in development but produces unexpected counts in production. What should be checked first?

A. Pipeline configuration, source freshness, filters, expectations, permissions, and run/event logs
B. Change all schemas to strings
C. Delete all checkpoints without investigation
D. Assume the platform is always wrong

Answer: A — Compare configuration and evidence across environments before making changes.

Q38. [Easy] Which is a sensible responsibility for a pipeline definition?

A. Specify source, transformations, target datasets, and data-quality rules
B. Store user passwords in source code
C. Replace access governance
D. Guarantee business-key uniqueness automatically

Answer: A — Pipeline definitions express data flow and quality logic; secrets and governance are handled separately.

Q39. [Difficult] A pipeline update would break downstream consumers because a column is renamed. What is the best approach?

A. Assess schema compatibility, coordinate the change, test consumers, and deploy through change control
B. Rename it directly in production with no notice
C. Drop all downstream tables
D. Disable schema checks permanently

Answer: A — Schema changes are interface changes and should be reviewed and validated end to end.

Q40. [Medium] What should happen when a quality expectation is configured to fail the update and a record violates it?

A. The update should fail according to that configured policy and the issue should be investigated
B. The record is always silently accepted
C. All source data is permanently deleted
D. Permissions are automatically expanded

Answer: A — Expectation behavior depends on its configured action; failure should be visible and diagnosable.

Unity Catalog

Q41. [Easy] What is Unity Catalog primarily used for in Databricks?

A. Centralized data and AI asset governance, access control, and discovery
B. Compressing Parquet files
C. Replacing Spark executors
D. Creating only dashboards

Answer: A — Unity Catalog centralizes governance for supported data and AI assets.

Q42. [Easy] Which is the usual three-level naming structure for a Unity Catalog table?

A. workspace.database.file
B. catalog.schema.table
C. cluster.folder.column
D. user.role.table

Answer: B — A fully qualified table name is commonly catalog.schema.table.

Q43. [Easy] What does the principle of least privilege mean?

A. Grant only the permissions needed for a user's task
B. Give everyone ownership
C. Share one admin account
D. Disable permission checks

Answer: A — Least privilege limits access to what is necessary.

Q44. [Medium] A user can query a table but cannot read its underlying external storage location through another path. Is this necessarily a bug?

A. Yes, all paths must be accessible to everyone
B. No; governed table access and direct storage access can be controlled differently
C. Yes, Unity Catalog disables SQL
D. No, because all storage is public

Answer: B — Governed access can intentionally differ between table privileges and direct storage permissions.

Q45. [Medium] What is an external location generally used to define?

A. A governed association between a cloud storage path and a storage credential
B. A Spark SQL alias only
C. A dashboard filter
D. A Delta version number

Answer: A — External locations help govern access to cloud storage paths using storage credentials.

Q46. [Medium] A new analyst needs SELECT access but must not modify a table. Which grant best matches the need?

A. SELECT
B. MODIFY and OWN
C. ALL PRIVILEGES
D. CREATE CATALOG

Answer: A — Grant only the read permission required; avoid broad privileges.

Q47. [Difficult] A notebook contains a cloud storage account key in plain text. What is the best corrective action?

A. Move secrets to an approved secret-management mechanism, rotate the exposed key, and review access
B. Commit the notebook to a public repository
C. Add the key to a table comment
D. Give every user access to the key

Answer: A — Exposed credentials should be rotated and stored securely, with access restricted.

Q48. [Difficult] A table is visible to one team but not another. Which is the most appropriate investigation?

A. Check identity, grants on catalog/schema/table, group membership, and applicable policies
B. Change all data to CSV
C. Disable audit logs
D. Rename the table randomly

Answer: A — Access is determined by identities, inherited/explicit privileges, and governance policies.

Q49. [Difficult] A service principal needs to read one governed table for a scheduled job. Which approach is best?

A. Grant the service identity only the necessary catalog/schema/table and compute permissions
B. Use a developer's personal password
C. Make the catalog public
D. Grant account admin for convenience

Answer: A — Use an appropriate non-human identity and least-privilege grants.

AI / ML / LLM

Q50. [Easy] What is the main purpose of MLflow in a machine-learning workflow?

A. Track experiments and help manage the ML lifecycle, including models
B. Replace all data storage
C. Schedule SQL joins only
D. Grant Unity Catalog privileges automatically

Answer: A — MLflow supports experiment tracking and model lifecycle capabilities; available integrations depend on setup.

Q51. [Easy] What is an ML experiment run commonly used to record?

A. Parameters, metrics, artifacts, and related model information
B. Only a user's password
C. Only the cluster's display color
D. Unrelated emails

Answer: A — Tracking runs enables comparison and reproducibility.

Q52. [Medium] A model performs well on training data but poorly on unseen data. What is the likely issue?

A. Overfitting
B. ACID compliance
C. Data masking
D. File compaction

Answer: A — Overfitting occurs when the model learns training-specific patterns that do not generalize.

Q53. [Medium] Why should an LLM application avoid sending confidential customer data to an unapproved model endpoint?

A. It may expose sensitive data and violate governance or contractual requirements
B. It always improves accuracy
C. LLMs cannot process text
D. It disables Delta transactions

Answer: A — Data handling must follow approved privacy, security, and compliance requirements.

Q54. [Easy] What is a large language model (LLM) primarily designed to do?

A. Process and generate language-like content from learned patterns
B. Enforce primary keys in Delta tables
C. Replace cloud storage
D. Automatically grant table privileges

Answer: A — LLMs generate or transform text and can support tasks such as summarization and question answering.

Q55. [Medium] What is retrieval-augmented generation (RAG)?

A. Retrieve relevant external/contextual information and provide it to an LLM to ground its response
B. Randomly increase model temperature
C. Delete training data
D. A Delta file compaction command

Answer: A — RAG combines retrieval with generation so responses can use relevant source material.

Q56. [Difficult] A RAG chatbot cites irrelevant documents even though the LLM prompt is clear. What should be investigated first?

A. Retrieval quality: chunking, embeddings, filters, ranking, and source permissions
B. Notebook font
C. Delta VACUUM retention only
D. Number of SQL comments

Answer: A — Poor grounding often originates in retrieval, indexing, chunking, ranking, or access-filtering.

Q57. [Difficult] Which practice best supports responsible evaluation of an LLM application?

A. Test representative prompts, factuality, safety, privacy, latency, and failure cases
B. Test only one easy prompt
C. Assume all generated text is correct
D. Remove logging and feedback

Answer: A — LLM evaluation needs broad, task-specific tests and ongoing monitoring.

Q58. [Medium] Why should an ML model's input schema and feature definitions be tracked?

A. To support reproducibility and detect training-serving mismatches
B. To make the model immune to drift
C. To eliminate evaluation
D. To avoid version control

Answer: A — Tracking inputs and features helps reproduce training and keep serving consistent.

Data Masking

Q59. [Easy] What is data masking intended to do?

A. Limit exposure of sensitive values while preserving permitted use
B. Increase data volume
C. Replace the transaction log
D. Speed up every query automatically

Answer: A — Masking reveals less-sensitive representations to users who should not see the original values.

Q60. [Easy] A support analyst needs to see the last four digits of a phone number but not the full number. Which approach fits?

A. Dynamic masking or a governed view that exposes only approved digits
B. Grant unrestricted access to the raw table
C. Copy all numbers into a public file
D. Disable auditing

Answer: A — Expose only the minimum necessary data through an approved masking or projection mechanism.

Q61. [Medium] Why is replacing sensitive data with a constant such as 'XXXX' not always equivalent to anonymization?

A. Other fields or external information may still re-identify a person
B. It always makes data public
C. It increases table uniqueness
D. It automatically encrypts backups

Answer: A — Masking one field does not eliminate re-identification risk from combinations of attributes.

Q62. [Medium] Which is the strongest way to apply consistent masking across governed tables for different users?

A. Centralized policy-based controls supported by the platform and reviewed access grants
B. Ask each analyst to remember not to view columns
C. Create unmanaged copies for everyone
D. Hide the column name in a dashboard only

Answer: A — Centralized governance is more consistent than relying on user behavior or cosmetic UI hiding.

Q63. [Difficult] A user has SELECT on a table containing a sensitive column. A dashboard hides that column, but direct SQL still returns it. What does this show?

A. UI hiding alone is not a security control; enforce masking/access policies at the data layer
B. The table is automatically encrypted
C. SQL cannot be governed
D. The user should receive admin rights

Answer: A — Security must be enforced by the governed data platform, not only by a dashboard layout.

Q64. [Difficult] A masked value is needed for analytics, but a small approved group may access the original value. What design principle applies?

A. Role-based least privilege with centrally enforced access and auditing
B. Put the original value in a shared notebook
C. Grant all users ownership
D. Disable all controls for convenience

Answer: A — Different access levels should be enforced with permissions/policies and monitored.

Q65. [Medium] Which practice is NOT a reliable data-masking control?

A. Hiding a sensitive field only in a chart while leaving unrestricted SQL access
B. Applying governed policies at the data layer
C. Auditing sensitive data access
D. Restricting privileges by role

Answer: A — Presentation-layer hiding does not prevent direct queries.

Compliance

Q66. [Easy] Why are audit logs important in a governed data platform?

A. They help establish who did what and when
B. They automatically fix every data defect
C. They replace backups
D. They eliminate access control

Answer: A — Auditing supports accountability, investigation, and compliance evidence.

Q67. [Medium] A teammate asks you to use another employee's credentials to access a restricted table. What should you do?

A. Refuse and request access through the approved process
B. Use the credentials if the task is urgent
C. Copy the data to a personal drive
D. Disable the permission check

Answer: A — Shared credentials and permission bypasses violate security practices; use authorized access.

Q68. [Medium] Which approach best supports data retention requirements?

A. Define and enforce approved retention/deletion policies, with legal and operational requirements considered
B. Keep all data forever by default
C. Delete all data immediately
D. Turn off history without checking dependencies

Answer: A — Retention must balance policy, law, audit, recovery, and platform behavior.

Q69. [Difficult] A data subject requests deletion, but records are copied into several downstream tables. What should the team do?

A. Follow the approved data inventory and deletion workflow, identify downstream copies, validate completion, and retain required audit evidence
B. Delete one row from the source and assume all copies disappear
C. Ignore downstream systems
D. Disable lineage

Answer: A — Deletion obligations require tracing copies and validating the full workflow, subject to applicable legal retention rules.

Q70. [Difficult] A developer wants to log complete customer records to debug a pipeline. What is the safer choice?

A. Log non-sensitive identifiers and diagnostic metadata, redacting sensitive payloads
B. Log all personal data in a public notebook
C. Disable access controls
D. Send records to personal email

Answer: A — Minimize sensitive data in logs while retaining enough diagnostic context.

Q71. [Difficult] A pipeline exports personal data to a new region. What should be assessed before deployment?

A. Data residency, contractual/legal requirements, access, encryption, and approved transfer controls
B. Only the notebook title
C. Whether the data has enough rows
D. Whether the SQL is lowercase

Answer: A — Cross-region transfers can trigger legal, contractual, and governance requirements.

SQL

Q72. [Easy] Which clause filters rows before grouping and aggregation?

A. WHERE
B. HAVING
C. ORDER BY
D. DISTINCT

Answer: A — WHERE filters input rows before GROUP BY; HAVING filters groups after aggregation.

Q73. [Easy] Which clause is used to filter aggregated groups, such as COUNT(*) > 1?

A. WHERE
B. HAVING
C. FROM
D. LIMIT

Answer: B — HAVING applies conditions to grouped/aggregated results.

Q74. [Easy] Which query pattern finds duplicate order IDs?

A. SELECT order_id FROM orders GROUP BY order_id HAVING COUNT(*) > 1
B. SELECT DISTINCT order_id FROM orders
C. SELECT order_id FROM orders WHERE COUNT(*) > 1
D. DELETE FROM orders

Answer: A — Group by the business key and filter groups with more than one row.

Q75. [Medium] You need to keep the latest row per customer. Which SQL pattern is best?

A. ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY updated_at DESC)
B. COUNT(*) without GROUP BY
C. ORDER BY customer_id only
D. SELECT * without ordering

Answer: A — ROW_NUMBER assigns an order within each customer group; filter rn = 1 to keep the latest row.

Q76. [Medium] What is the main difference between UNION and UNION ALL?

A. UNION removes duplicate result rows; UNION ALL preserves them
B. UNION ALL always sorts results
C. UNION works only on Delta tables
D. There is no difference

Answer: A — UNION typically performs duplicate elimination; UNION ALL keeps all rows and is often cheaper.

Q77. [Medium] A LEFT JOIN returns fewer rows than expected because a filter on the right table is placed in WHERE. Why can this happen?

A. The WHERE filter can eliminate unmatched rows and effectively change outer-join behavior
B. LEFT JOIN always removes left rows
C. WHERE runs before FROM
D. SQL does not support nulls

Answer: A — A right-side WHERE condition can reject NULL-extended rows; predicate placement matters.

Q78. [Difficult] A query joins orders to a customer table that has multiple rows per customer_id. Revenue is inflated. What is the likely cause?

A. Join cardinality multiplies order rows; deduplicate or model the customer key correctly before joining
B. SUM cannot aggregate numbers
C. GROUP BY always doubles rows
D. SQL aliases cause duplication

Answer: A — A many-to-many or one-to-many join can multiply facts and inflate aggregates.

Q79. [Difficult] You need to find records with no matching customer. Which pattern is appropriate?

A. LEFT JOIN customers c ON ... WHERE c.customer_id IS NULL
B. INNER JOIN customers
C. GROUP BY every column
D. ORDER BY NULL

Answer: A — A left anti-join pattern identifies rows without a matching right-side key.

Q80. [Difficult] A column contains NULL. Which condition correctly tests for missing values?

A. column IS NULL
B. column = NULL
C. column == NULL in standard SQL
D. NULL(column) = 0

Answer: A — SQL NULL uses IS NULL/IS NOT NULL rather than equality comparison.

Architecture / Cluster Scenarios

Q81. [Easy] In Spark, which component coordinates the application and schedules tasks?

A. Driver
B. Executor only
C. Delta log
D. Unity Catalog schema

Answer: A — The driver coordinates execution; executors run tasks on data partitions.

Q82. [Medium] A 5 GB DataFrame must be joined with a 50 MB lookup table. What is a reasonable option to test?

A. Broadcast the smaller side if it fits executor memory, then inspect the physical plan
B. Collect both full tables to the driver
C. Always disable AQE
D. Convert both to Python lists

Answer: A — Broadcast can avoid a large shuffle, but memory, thresholds, skew, and explain() should be checked.

Q83. [Medium] Which operation normally causes a full shuffle?

A. repartition()
B. withColumn() with a simple expression
C. select()
D. filter()

Answer: A — Repartition generally redistributes records across partitions and incurs a shuffle.

Q84. [Medium] When is coalesce(n) commonly preferred over repartition(n)?

A. When reducing partitions and avoiding a full shuffle is desirable
B. When increasing partitions evenly is required
C. When joining two tables by key
D. When masking columns

Answer: A — Coalesce is commonly used to reduce partition count with less data movement, though skew may remain.

Q85. [Difficult] A job fails with driver out-of-memory during collect(). What is the best response?

A. Avoid collecting the full dataset; keep processing distributed or collect only a small validated result
B. Increase driver memory without checking the code
C. Convert the entire dataset to a Python list
D. Disable all transformations

Answer: A — collect() moves all result rows to driver memory and can exhaust it.

Q86. [Difficult] One Spark task takes far longer than all others and processes a disproportionately large partition. What is a likely issue?

A. Data skew
B. Column masking
C. SQL capitalization
D. Time travel

Answer: A — Skew causes uneven work across partitions; inspect key distribution and consider skew-aware strategies.

Q87. [Medium] Why should you inspect explain() for a join optimization?

A. To verify the physical plan and whether the intended join strategy was selected
B. To grant permissions
C. To show masked values
D. To change the source schema automatically

Answer: A — The physical plan reveals join strategy and other execution choices.

Q88. [Difficult] A production cluster terminates mid-job. What should a junior engineer do first?

A. Check permitted run/cluster logs and termination reason, assess impact, and escalate before changing production
B. Restart and resize it without approval
C. Delete the job
D. Share admin credentials

Answer: A — Investigate with authorized access, document evidence, and follow incident procedures.

Q89. [Medium] What is the main difference between a Spark transformation and an action?

A. Transformations build a lazy plan; actions trigger execution or return results
B. Actions are always lazy and transformations are not
C. Both always execute immediately
D. Transformations grant permissions

Answer: A — Spark transformations are lazy; actions such as count(), collect(), and write trigger computation.

Final revision: high-value rules to remember

Lakeflow: Model dependencies explicitly; inspect run/task logs; retry only after checking partial writes and idempotency.

Delta Lake: Delta adds transaction-log semantics to Parquet; it does not automatically guarantee unique business keys. Validate MERGE sources.

CDC: Handle ordering, duplicate events, deletes, late arrivals, and watermark/checkpoint recovery.

SDP: Declare datasets and rules clearly; make expectations and invalid-record handling observable.

Unity Catalog: Use catalog.schema.table, least privilege, governed storage access, and secure identities/secrets.

AI/ML/LLM: Track experiments, evaluate generalization and RAG retrieval, and protect sensitive data.

Masking/compliance: UI hiding is not security. Enforce controls at the data layer, audit access, and follow approved retention/deletion procedures.

SQL: WHERE filters rows; HAVING filters groups. Use ROW_NUMBER for deterministic latest-row logic and validate join cardinality.

Spark architecture: Driver coordinates; executors process partitions. Avoid large collect(); inspect explain(), skew, shuffle, and memory evidence.

Production behavior: Triage, document evidence, assess impact, escalate, obtain approval, validate in lower environments, and monitor/rollback.

Suggested exam practice: first attempt only the questions matching the official easy/medium/difficult distribution, then retake all questions without looking at explanations. For full readiness, practise the separate coding section too; this document focuses on MCQs as requested.